

Topic 12

Docker + Deployment + CI


Dockerfile


Docker Compose


Env Config


GitHub Actions CI


Mini Project

Dockerfile định nghĩa cách đóng gói app thành image. Dùng multi-stage build để image nhỏ gọn, không đưa devDeps vào production.

Dockerfile (multi-stage build)

```
# — Stage 1: builder —————
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

# — Stage 2: production —————
FROM node:20-alpine AS production
WORKDIR /app
ENV NODE_ENV=production
COPY package*.json ./
RUN npm ci --omit=dev # no devDeps
COPY --from=builder /app/dist ./dist

USER node # non-root user ✓
EXPOSE 3000
CMD ["node", "dist/index.js"]
```

✓ Best practices

- Multi-stage: image prod chỉ có dist/ + prod deps
- COPY package*.json trước → cache layer npm install
- USER node — không chạy root trong container
- Alpine base (~50MB vs ~900MB full node image)
- EXPOSE chỉ để document — không bind port thật

.dockerignore

```
node_modules
dist
.env* # không đưa secrets vào image!
coverage
*.test.ts
.git
README.md
```

docker-compose.yml định nghĩa toàn bộ local stack (API + DB + Redis). Một lệnh `docker compose up` → môi trường chạy hoàn chỉnh.

docker-compose.yml

```
services:
  api:
    build: .
    ports:
      - "3000:3000"
    env_file:
      - .env.development
    depends_on:
      db:
        condition: service_healthy
      redis:
        condition: service_started
    volumes:
      - ./src:/app/src # hot reload

  db:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: appdb
      POSTGRES_USER: app
      POSTGRES_PASSWORD: secret
    volumes:
      - pg_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U app"]
      interval: 5s

  redis:
    image: redis:7-alpine

volumes: { pg_data: {} }
```

Lệnh hay dùng

<code>docker compose up -d</code>	Chạy background
<code>docker compose up --build</code>	Rebuild image rồi chạy
<code>docker compose logs -f api</code>	Tail logs của service api
<code>docker compose exec api sh</code>	Shell vào container
<code>docker compose down -v</code>	Dừng + xoá volumes

docker-compose.override.yml (dev only)

```
# Tự động load khi compose up
services:
  api:
    command: npm run dev # nodemon
    environment:
      DEBUG: "app:*"
    volumes:
      - ./src:/app/src
# Production: compose -f compose.yml up
```

Config nằm trong environment, không trong code. Mỗi môi trường có file .env riêng. KHÔNG commit secret vào git.

Chiến lược env files

<code>.env.example</code>	Template — COMMIT vào git, không có giá trị thật
<code>.env.development</code>	Local dev — KHÔNG commit (thêm vào .gitignore)
<code>.env.test</code>	Dùng cho vitest/jest — DB test riêng biệt
<code>.env.production</code>	Server inject qua CI/CD secrets — không có file này

src/config/index.ts — typed config loader

```
import { z } from 'zod'

const schema = z.object({
  NODE_ENV: z.enum(['development', 'test', 'production']),
  PORT: z.coerce.number().default(3000),
  DATABASE_URL: z.string().url(),
  JWT_SECRET: z.string().min(32),
  REDIS_URL: z.string().url(),
});

export const config = schema.parse(process.env);
// Throws at startup if any var missing ✓
```

.env.example

```
# App
NODE_ENV=development
PORT=3000
LOG_LEVEL=info
# Database
DATABASE_URL=postgresql://user:pass@db:5432/appdb
# Auth
JWT_SECRET=change_me_in_production
JWT_EXPIRES_IN=15m
# Redis
REDIS_URL=redis://redis:6379
```

🔒 Secrets — KHÔNG bao giờ

- ✗ Commit .env vào git (dù là private repo)
- ✗ Hardcode secret trong source code
- ✗ Log giá trị của JWT_SECRET, DB password
- ✗ Dùng cùng secret cho dev và production
- ✓ Dùng GitHub Secrets / AWS SSM / Vault
- ✓ .gitignore chứa .env, .env.*, !.env.example

CI chạy tự động khi push/PR. Pipeline cơ bản: checkout → install → lint → test. Fail fast — không merge nếu CI errors.

.github/workflows/ci.yml

```
name: CI
on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main, develop]
jobs:
  ci:
    runs-on: ubuntu-latest
    services:
      postgres: # spin up postgres
        image: postgres:16-alpine
        env:
          POSTGRES_DB: testdb
          POSTGRES_USER: test
          POSTGRES_PASSWORD: test
        options: >-
          --health-cmd pg_isready
          --health-interval 5s
          --health-retries 5
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: 'npm'
      - run: npm ci
      - run: npm run lint
      - run: npm run build
      - run: npm test
    env:
      DATABASE_URL: postgresql://test:test@localhost:5432/testdb
      JWT_SECRET: ci_secret_min32chars_fortest
```

Pipeline stages



Branch protection rules

- ✓ Require CI pass trước khi merge
- ✓ Require ≥ 1 code review approval
- ✓ Restrict push thẳng lên main

package.json scripts

```
"lint" : "eslint src --max-warnings 0"
"build" : "tsc --noEmit"
"test" : "vitest run --coverage"
```

Lấy API từ tuần trước → đóng gói bằng Docker, chạy full stack qua docker compose, CI tự động lint + test khi push.

Yêu cầu kỹ thuật

- ▶ Dockerfile multi-stage (builder + production)
- ▶ docker-compose.yml: api + postgres + redis
- ▶ docker compose up → app chạy ở localhost:3000
- ▶ npm run migrate chạy được trong container
- ▶ .env.example đủ các biến, .dockerignore đầy đủ
- ▶ GitHub Actions: lint → build → test (với postgres service)
- ▶ npm test pass toàn bộ trên CI (không chỉ local)
- ▶ README: hướng dẫn setup từ đầu bằng 3 lệnh

README — 3 lệnh setup từ đầu

```
# 1. Clone & copy env
cp .env.example .env.development

# 2. Start full stack
docker compose up --build -d

# 3. Run migrations
docker compose exec api npm run migrate
```

✓ Checklist done khi nào?

- ☐ docker compose up --build không báo lỗi
- ☐ curl localhost:3000/health → 200 OK
- ☐ docker image ls | grep api → ≤ 200MB
- ☐ CI workflow xanh trên GitHub Actions
- ☐ PR lên main bị block nếu CI đỏ
- ☐ .env không bị commit vào git
- ☐ docker compose down -v → up lại vẫn migrate OK
- ☐ README: 3 lệnh để setup từ đầu

💡 Gợi ý triển khai

Image size	docker build --no-cache → kiểm tra size
CI secret	Dùng GitHub Secrets cho JWT_SECRET
Wait for DB	depends_on condition: service_healthy
Cache npm	actions/setup-node cache: 'npm' tăng tốc 2x